

Taller Express y otras cosas



Nombre: Carlos Daniel Arias Aispuro

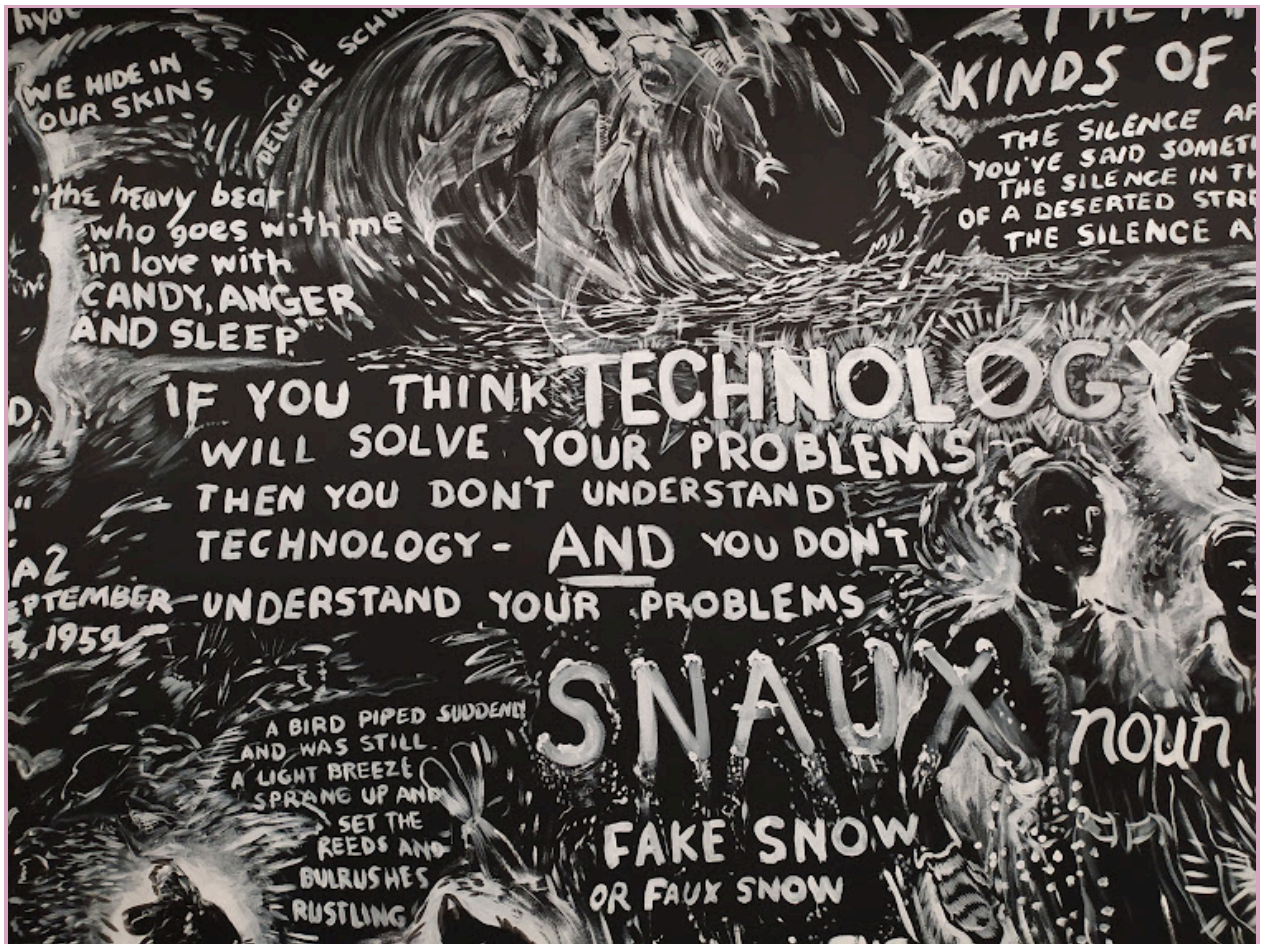
Matrícula: A01743621

¿Quién eres?: Soy un estudiante de ITC

El objetivo de la actividad es explorar y conocer más sobre el desarrollo web del lado del backend.

Al final de la actividad deberás hacer un PDF con este documento. El PDF deberá tener un enlace en [miarchivo.html](#)

- **Ejercicio -1: Una pregunta**



¿Qué piensas de la frase?

Se me hace una frase con un buen sentido, porque es cierto que muchas veces la tecnología nos ayuda a resolver varios problemas, sin embargo muchas veces nos vamos de paso al querer resolver todo usando esta herramienta, esto no siempre va a resultar bien, en demasiadas ocasiones nos terminará dando más problemas, por eso uno debe de saber bien cuales son sus problemas, para poder así llegar a una solución, la cual puede o no llegar a facilitarse con la tecnología.

¿Qué problema crees que podrías resolver con tecnología?

Tener trabajo, comunicarse a distancia, estar informado, etc.

• Ejercicio 0: El pasado

En las semanas previas vimos la forma de consumir un API desde el cliente (el navegador). Ya consumiste el **API desde el servidor** usando javascript y nodejs.

Escribe aquí abajo el enlace al archivo (con esa solución) en tu repositorio:

https://github.com/CarlosArias123/CarlosArias123.github.io/blob/main/etapa_0.js

¿Cuál fue el principal problema para resolver este ejercicio?

Encontrar una API accesible la cual contenga su respectiva API Key y que esta también maneje una buena documentación para no confundirme al implementarla.

¿Cuál fue el problema que tuvo una de las personas en tu equipo? Explica cómo resolvió esa persona ese problema.

Al parecer estaba mal la documentación de una API, gracias a esto se perdió bastante tiempo, pudiendo resolverlo solo cambiando de API.

• Ejercicio 1: Simplicidad

¿ Recuerdas cómo **instalar un módulo** en node.js? Escribe la instrucción que se usa: Escribe en la terminal **npm install “Librería”**

• Ejercicio 2: Express es

Investiga qué es express (en el contexto de desarrollo web en nodejs). Explica a un **niñ@** qué es express, cómo se usa, para qué sirve. Escribe ese texto aquí. Incluye también **las referencias** de tu investigación.

Express es una herramienta que te ayuda a complementar otras aplicaciones para hacer páginas web.

Dile a una persona de la clase (que no esté en tu mesa) que te explique qué es express y qué ventajas te podría dar su uso. Graba el audio de la explicación.

¿Crees que realizó una buena explicación? ¿Por qué?

• Ejercicio 3: Desde un libro

Investiga una definición en un libro (digital o físico) **de una API**. Incluye la referencia del libro.

Según el libro RESTful Web APIs, una API es un conjunto de reglas que permite que diferentes sistemas se comuniquen entre sí mediante solicitudes y respuestas.

Escribe un ejemplo de un endpoint de tu proyecto.
`http://localhost:1984/post`

• Ejercicio 4: Instala express

Escribe la instrucción que se usa para **instalar el módulo de express**:

```
npm install express
```

• Ejercicio 5: Uso de express

Ejecuta y explica qué hace el siguiente código:

```
import express from 'express';

const app = express();

app.listen(1984, () => {
  console.log('Up and up');
});
```

Importa la librería express, la cual nos ayuda a facilitar el despliegue de una página web; al final del código una vez que ya esté el servidor corriendo, se imprime en la consola “Up and up”.

• Ejercicio 6: Uso de express++

Ejecuta y explica qué hace el siguiente código (agregado al ejercicio 5):

```
app.get('/bienvenida', (req, res) => {  
  res.send('Esto no es una página html');  
});  
  
app.get('/otraBienvenida', (req, res) => {  
  res.sendFile('bienvenida.html');  
});
```

En caso de que alguno de estos te marque error, soluciona el problema y explica la forma de solucionarlo:

app.get('/bienvenida... Define un GET llamado /bienvenida la cual solo nos da un texto.

app.get('/otraBienvenida... Define un GET el cual envía un archivo llamado "bienvenida.html".

Lo resolvimos poniendo la ruta de dónde está guardado bienvenida.html y metiendo un if que nos avise si hay un error porque el archivo no se encontró o no se mandó bien.

● Ejercicio 7: Recuerdo de imagen

Es posible si pusiste una imagen en el servidor.js no se podía ver. Un servidor entrega respuestas a solicitudes realizadas por un cliente. Cuando el cliente solicita una imagen (u otro tipo de archivo) el servidor tiene que saber responder.

Express nos propone crear un directorio para los archivos que queremos usar, por ejemplo imágenes o archivos de hojas de estilo.

Investiga qué es express.static.

Escribe tu investigación

Básicamente, express.static es una herramienta que ya trae Express para que el navegador pueda ver nuestros archivos sin que tengamos que armar una ruta

manual para cada cosa. Nos sirve para compartir archivos estáticos, o sea: imágenes, los CSS, el JS del frontend, PDFs o puros HTMLs.

• Ejercicio 8: Canción

Pregunta a una persona de otro equipo su canción favorita y sus razones.

¿Ya la conocías? ¿Te gusta? ¿Por qué?

La línea (Julieta Venegas)

• Ejercicio 9: Transformación

Usa express para transformar todo el código de servidor.js (el que tenía lo de opinión), nombra el archivo ahora servidor_express.js

Haz comentarios de qué te parece *express* en el mismo código.

Haz que se vea una foto de un árbol de tu universidad con el nombre científico del árbol en una de tus páginas html.

Escribe aquí abajo el enlace al archivo (servidor_express.js) en tu repositorio:

https://github.com/CarlosArias123/CarlosArias123.github.io/blob/main/servidor_express.js

• Ejercicio 10: Verbos

En el contexto de desarrollo web, explica los siguientes verbos GET, POST, PUT, DELETE

GET: Es solo para pedir o leer datos. Es seguro porque no cambia ni rompe nada.

POST: Lo usamos para mandar información nueva y crear algo en el servidor (aquí sí hacemos cambios).

PUT: Nos sirve para actualizar algo que ya existe, pero sobrescribiendo toda su información.

DELETE: Es tal cual para borrar algo del servidor."

Explica los siguientes códigos, en el contexto de desarrollo web:

1xx: Significa que la solicitud sigue en proceso.

2xx: Significa que la solicitud salió bien.

3xx: Significa una redirección, como que puede hacer falta un paso extra o que nos tenemos que dirigir a otro sitio.

4xx: Significa un error del cliente, osea del que envía la petición.

5xx: Significa un error en el servidor, básicamente significa que hubo un error en el backend.

• Ejercicio 11: Postman

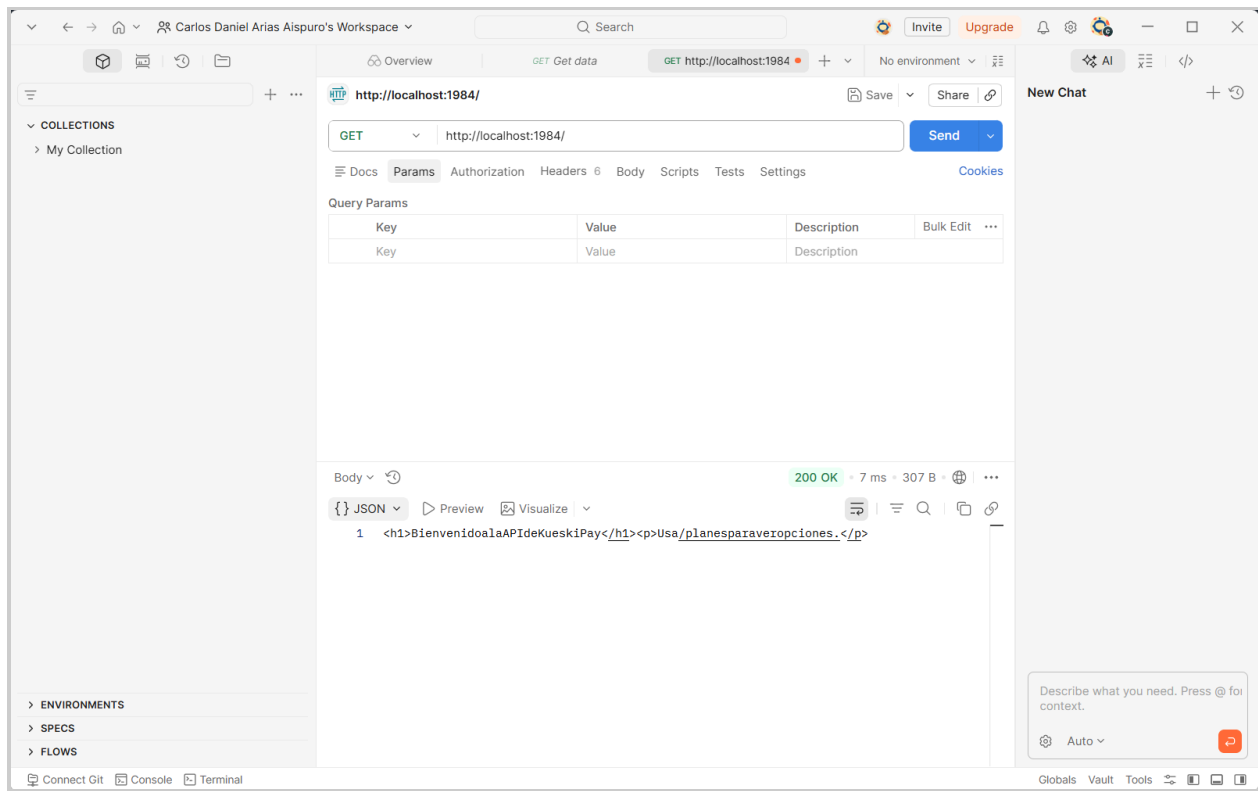
En equipo con tu mesa, instalen postman en cada una de sus computadoras. Tomen una misma foto en la que aparezcan todas las pantallas con postman abierto.

Foto aquí:

Explica para qué sirve Postman

Básicamente te deja probar tus rutas (mandando GET, POST y demás), ver las respuestas del backend y detectar fallas rápidamente, ahorrándote la necesidad de programar una interfaz gráfica desde cero.

Prueba lo que hiciste en el ejercicio 9 con Postman. Pon una captura de pantalla usando un GET.



• Ejercicio 12: Parámetros

Inventa un ejercicio (que esté relacionado con tu reto) para que uses parámetros en las rutas (por ejemplo /getBancos /postBancos)

Soluciona ese ejercicio

Escribe aquí abajo el enlace al archivo en tu repositorio:

https://github.com/CarlosArias123/CarlosArias123.github.io/blob/main/servidor_express.js

Compara ese ejercicio con el de otra persona de la clase. ¿Qué ejercicio ayuda mejor a comprender el uso de parámetros y por qué?

• Ejercicio 13: npx

Investiga con alguien más lo siguiente: `npx express-generator`. Importante haz un directorio diferente si quieres ejecutar el comando.

Escribe el nombre de ese alguien más:

Cristian Cárdenas

Explica lo que genera y hace ese comando.

Al correr el comando (como `npx express-generator mi-app`), te arma de golpe toda una carpeta ya con los archivos y subcarpetas que necesitas para arrancar.

Explica de manera básica y conceptualmente qué archivos se generaron.

app.js Configura el servidor.

bin/www Inicia la aplicación.

routes/ Define las rutas.

views/ Guarda las vistas (HTML dinámico).

public/ Aloja los archivos estáticos (CSS, imágenes).

package.json Lista las dependencias del proyecto.

• Ejercicio 14: Explicar

Explica a una niña o un niño de 10 años para qué sirven frameworks como Angular, React, Vue, o Svelte.

Hacer todo desde cero es un dolor de cabeza porque tienes que picar código hasta para que funcione un simple botón. Ahí es donde nos salvan los frameworks (como Angular, React, Vue o Svelte). Básicamente son kits de herramientas que ya te dan la base y un montón de cosas listas para usar, ahorrándote muchísimo tiempo de desarrollo.